WINDOWS · macOS · LINUX

py.

多版本 Python 与虚拟环境

从新电脑上的第一个解释器，到每个项目独立的 venv：讲清 py 启动器、PATH、pip、uv 与 conda 的边界，并给出可以照着执行的验收与排错清单。

WINDOWS 10 / 11

macOS 12+

LINUX



READ ME FIRST / 使用说明

关于本指南

新电脑上真正麻烦的，不是“下载一个 Python”，而是三个东西同时存在：多个解释器、多个项目环境、多个入口命令。装完之后，你必须能回答：现在敲的 `python` 是哪一只？这个项目的包装进了哪里？下次换电脑，怎么在十分钟内复现同样的环境？

DOCUMENT	BASELINE	UPDATED
Python 多版本与环境配置手册	**Python 3.15.2** 2026-10-25 发布	**2026 年 10 月** 版本以官网为准

适合谁读

- 刚拿到新电脑，准备从零搭 Python 开发环境的人。
- 已经装过多个版本，但 `python`、`pip`、`py` 经常对不上的人。
- 需要同时维护 3.11 / 3.13 / 3.15 项目的开发者。
- 要为公司或实验室写新机初始化文档的维护者。

本指南覆盖什么

- Windows、macOS、Linux 的解释器安装与多版本管理。
- py 启动器、Homebrew、pyenv、uv、conda 的取舍。
- venv 的原理、创建、激活、绑定与重建。
- pip、requirements、pyproject.toml、IDE 与验收排错。

怎么读最省时间

ROUTE 01

全新电脑，第一次装

读第 1–2 章建立模型，按系统跳到第 3、4 或 5 章，再照着第 10 章验收。

ROUTE 02

已经装好，但很乱

直接看第 1.2 节与第 3.3 节，先确认 PATH 和 `py -0p`，再清掉重复入口。

ROUTE 03

只关心虚拟环境

从第 6 章开始；第 7–8 章讲依赖与 IDE，第 11 章是报错索引。

ROUTE 04

团队 / 教学

附录 A 可打印，附录 B 是两个初始化脚本，附录 C 是版本与资源表。

01 **入口唯一**
知道每条命令实际调用谁

02 **项目隔离**
每个项目一个独立环境

03 **可以复现**
换机器能重建同样的依赖

命令与排版约定

写法	含义
`PS> py -0p`	Windows PowerShell 提示符，不输入 `PS>`。
`$ python3 --version`	macOS / Linux 终端提示符，不输入 `$`。
`<3.12>`	尖括号是占位符，替换为实际版本号时不要保留尖括号。
管理员	Windows 指“以管理员身份运行”；macOS / Linux 指命令前加 `sudo`。

开始之前：三条边界

不要为了省事把每个 Python 的根目录和 `Scripts` 都塞进 PATH；入口越多，`pip` 装错地方的概率越高。

不要直接升级系统自带的 Python（macOS 的 `/usr/bin/python3`、Linux 发行版的 `/usr/bin/python3`）。系统工具依赖它，用独立安装或 pyenv 管理新版本。

版本说明：本指南以 2026 年 10 月 25 日发布的 Python 3.15.2 为基线，适用于 Python 3.10 及以上。安装器界面、Homebrew 公式名与 pyenv 版本号会随发布时间变化，请把“版本号”当作示例，把“选项关键词”当作依据。

五个最常见的配置误区

误区	后果	在哪里解决
把 `python` 与 `py` 当成同一个东西	以为装了 3.13，实际一直运行 3.9。	1.2 节、3.2 节
每个 Python 都加进 PATH	`pip` 和 `python` 来自不同版本，包装错位置。	3.3 节、10.2 节
把 `.venv` 提交进 Git	仓库巨大、跨平台不可用、路径写死。	8.2 节
在 conda 环境里再建 venv	两套环境叠加，`python` 来源无法解释。	9.4 节、9.5 节
裸敲 `pip install`	装进当前 PATH 的 Python，而不是项目环境。	7.1 节

表 0-1 新电脑配置阶段的高频错误。它们大多不是“不会命令”，而是入口与环境没有对齐。

目录

CONTENTS / 12 CHAPTERS & APPENDICES

PART I 解释器与安装

01 **先理清：解释器、环境与入口** 7

1.1 三个层次，而不是三个软件 7

1.2 为什么新版本装了却“没生效” 8

1.3 新电脑的配置目标 8

1.4 四条路线怎么选 8

02 **安装前的版本与工具决策** 10

2.1 2026 年该装哪个版本 10

2.2 支持周期与 EOL 10

2.3 py、pyenv、uv、conda 怎么选 11

2.4 目录、权限与命名 11

03 **Windows：官方安装器与 py 启动器** 13

3.1 安装器逐项选择 13

3.2 py 启动器与多版本 14

3.3 PATH 与 Microsoft Store 别名 16

3.4 多个版本的安装顺序 16

3.5 验证、升级与卸载 17

04 **macOS：系统 Python、官方包与 Homebrew** 18

4.1 先看系统 Python 18

4.2 python.org 安装包 18

4.3 Homebrew 与 PATH 19

4.4 pyenv 多版本 20

4.5 验证与卸载 20

05 **Linux：系统包、deadsnakes 与 pyenv** 21

5.1 包管理器与 venv 支持 21

5.2 Ubuntu deadsnakes PPA 21

5.3 pyenv 编译安装 22

5.4 WSL、容器与最小化环境 23

5.5 不要动系统 python3 23

PART II 环境与依赖

06 venv：项目级隔离的核心 25
6.1 venv 到底做了什么 25
6.2 创建环境的正确命令 26
6.3 激活与不激活 26
6.4 解释器绑定与重建 27
6.5 常见选项与隔离边界 28
07 pip、依赖与可复现 29
7.1 永远用 python -m pip 29
7.2 升级与常用命令 30
7.3 requirements 的三种写法 30
7.4 pyproject.toml 与可编辑安装 31
7.5 镜像、代理与证书 32
7.6 依赖冲突与 pip check 33
08 项目工作流与 IDE 34
8.1 一个项目一个环境 34
8.2 目录结构与 .gitignore 34
8.3 VS Code、PyCharm 与 Jupyter 35
8.4 .env 与配置 36
8.5 初始化脚本 36
09 多版本并用：测试、切换与冲突 38
9.1 同一项目测多个版本 38
9.2 tox 与 nox 38
9.3 uv 工作流 39
9.4 conda 工作流 40
9.5 五个真实冲突案例 40

PART III 验收、排错与维护

10 验收与诊断 42
10.1 十二项验收清单 42
10.2 诊断命令与正常输出 42
10.3 网络、镜像与证书 43
11 常见报错 25 例 45
11.1 Windows 与 PATH 45
11.2 venv 与 pip 46
11.3 依赖、编译与 SSL 46
11.4 macOS 与 Linux 47
12 维护、安全与迁移 48
12.1 版本升级策略 48
12.2 供应链与镜像安全 48
12.3 迁移、备份与清理 49

APPENDICES

A 命令速查表 50
B 初始化脚本模板 52
C 版本、资源与术语 54

按问题找章节

你遇到的问题	先看	再查
`python` 版本不对 / 找不到	1.2、3.2	11.1
创建 venv 失败 / ensurepip	6.2	11.2
`pip` 装错环境 / 包找不到	7.1	11.2
VS Code / Jupyter 解释器不对	8.3	9.5
多个版本测试 / CI	9.1、9.2	9.3
镜像、代理、SSL 证书	7.5	10.3
换电脑后环境重建	12.3	附录 B

按“症状 → 机制 → 排错”顺序阅读，比从头翻更省时间。

页码为条目所在页；点击条目可跳转。目录在排版完成后自动生成。

FOUNDATION / 先建立模型

01

先理清：解释器、环境与入口

多版本 Python 的混乱，几乎都能归结为一句话：你以为自己在切换“Python”，实际切换的可能是解释器、虚拟环境、PATH 顺序或包管理器中的某一个。先把这三个层次分开，后面的安装与排错才不会互相打架。

预计 15 分钟 | 无需安装 | 概念与路线

1.1 三个层次，而不是三个软件

一台电脑上可以同时存在 Python 3.11、3.12、3.13、3.14、3.15，也可以同时存在 venv、conda 环境、uv 环境。它们不是同一种东西，解决的问题也不同：

层次	解决的问题	典型工具
解释器	机器上有哪些 Python 版本，每个版本的可执行文件在哪里。	python.org 安装器、`py`、Homebrew、pyenv、uv、conda
项目环境	同一个版本下，A 项目与 B 项目的依赖如何互不污染。	`venv`、`conda env`、`uv venv`
入口	你敲下的 `python`、`pip`、`jupyter` 实际指向哪一个文件。	PATH、`py` 启动器、shell 激活脚本、IDE 解释器设置

表 1-1 三个层次。安装新版本只改变第一层；没有创建环境，项目依赖仍然装在全局。



举一个最常见的场景：你装了 Python 3.13，但终端里的 `python` 仍然是 3.9。原因通常不是安装失败，而是：3.9 在 PATH 中排在前面，或者 3.13 根本没有被加入 PATH。此时 `py -3.13` 可能是可用的，但 `python` 不是它。

> **一个判断习惯**
>
> 看到“Python 版本不对”时，先问三件事：`python` 是哪个文件？它属于哪个环境？这个环境是用哪个解释器创建的？不要急着重装。

1.2 为什么新版本装了却“没生效”

Windows 上，解释器的查找顺序由 PATH 决定；macOS / Linux 上则由 shell 的 PATH 与符号链接共同决定。安装器把文件放进了磁盘，不等于终端会优先找到它。

WINDOWS

两个常见入口

`py` 是启动器，按版本号选择解释器；`python` 是 PATH 中第一个 `python.exe`。两者可以指向不同版本，这是设计如此，不是 bug。

macOS / LINUX

符号链接与 PATH

`python3` 往往是一个符号链接；Homebrew、pyenv、系统包管理器都可能改它。先看 `which -a python3`，再看链接指向。

还有一种更隐蔽的情况：你在 IDE 里选了一个解释器，但集成终端没有激活对应环境。编辑器右上角显示 3.13，终端里运行 `python` 却是 3.9。这时代码检查可能正常，运行却报 `ModuleNotFoundError`。第 8 章会专门讲 IDE 的解释器与终端如何对齐。

现象	最可能的原因	先查什么
`python --version` 是旧版	PATH 前面有旧解释器	`where python` / `which -a python3`
`py -3.13` 能用，`python` 不能	`py` 已注册，但 3.13 没进 PATH	`py -0p`，不需要把 3.13 加进 PATH
`pip install` 后仍找不到包	`pip` 与运行代码的 `python` 不是同一个	`python -m pip --version`、`sys.executable`
终端里是 3.13，IDE 里是 3.9	IDE 解释器与终端环境不一致	IDE 的 Python Interpreter 设置

表 1-2 “版本没生效”的四种典型结构。先定位入口，再决定要不要重装。

1.3 新电脑的配置目标

一份合格的新电脑 Python 环境，不是“装了很多版本”，而是满足下面四个目标。后文章节都围绕它们展开。

- [ ] **入口可解释。**每个 `python`、`pip`、`py` 都能说出它来自哪里。
- [ ] **项目可隔离。**每个项目有独立 `.venv`，不使用全局 site-packages。
- [ ] **升级可回滚。**知道当前版本、EOL 时间与回退方式。
- [ ] **版本可共存。**至少保留一个稳定版与一个较新版本，互不覆盖。
- [ ] **依赖可复现。**换电脑能根据 `pyproject.toml` 或锁定文件重建。
- [ ] **排错有证据。**能提供 `sys.executable`、PATH 与 pip 来源，而不是只说“环境坏了”。

1
个默认入口（推荐 `py` 或 `python3`）

2–3
个长期保留的解释器版本

N
个项目 = N 个独立环境

1.4 四条路线怎么选

不同系统、不同使用习惯适合不同工具。下面不是“哪个最好”，而是“哪个更适合你现在这台机器”。

路线	适合	优点	代价
官方安装器 + py	Windows 新手、需要稳定基线	原生、直观、每个版本独立目录；`py` 负责选择	安装器较多；PATH 需要克制
Homebrew / 系统包	macOS、Linux 日常开发	升级方便，系统集成好	版本受仓库影响；多版本链接容易冲突
pyenv	需要频繁切换 3.10–3.15	按项目固定版本，版本齐全	需要编译依赖；shell 初始化要配好
uv	追求速度、新项目、CI	装解释器与建环境都很快，锁文件现代	新工具，团队需要统一约定
conda	数据科学、CUDA、非 Python 依赖	二进制包与系统库省心	体积大；不要与 venv 叠加

表 1-3 路线对比。可以组合：Windows 用 `py` 管解释器，项目内仍用 `venv`。

推荐的新电脑基线

Windows：官方安装器装 3.13 与 3.15，保留 `py` 启动器，项目统一 `py -3.x -m venv .venv`。**macOS：**Homebrew 装一个主版本，pyenv 或 uv 管其他版本。**Linux：**系统包保证 `python3-venv`，需要新版本用 deadsnakes 或 pyenv，绝不动系统解释器。

DECISIONS / 安装之前

02

安装前的版本与工具决策

新电脑最容易犯的错，是“看到最新版就装最新版”。Python 的版本选择不只是语法新旧，还涉及轮子（wheel）是否齐全、系统库是否兼容、团队是否统一。先做四个决定，再下载安装包。

预计 18 分钟 | 版本与 EOL | 工具对比

2.1 2026 年该装哪个版本

截至 2026 年 10 月 25 日，Python 最新稳定版是 **3.15.2**；3.14.10、3.13.18、3.12.16、3.11.18 仍在维护。3.10 已于 2026 年 10 月 1 日结束支持。不要再用 3.10 开新项目。

版本	状态（2026-10）	建议用途	说明
3.15	最新稳定，bugfix	尝鲜、个人新项目	新特性最多；部分二进制轮子可能还没跟上，生产项目先验证依赖。
3.14	稳定，bugfix	**新项目默认候选**	发布满一年，生态适配较好；支持到 2030-10。
3.13	安全维护	稳妥的生产基线	轮子覆盖广，支持到 2029-10；适合作为团队主版本。
3.12	安全维护	兼容旧项目	支持到 2028-10；新项目不必优先选。
3.11	安全维护	维护旧项目	支持到 2027-10，已进入后半程。
3.10	**已 EOL**	不建议	2026-10-01 结束支持，不再有安全修复。

表 2-1 版本选择。新电脑建议装 3.13 或 3.14 作为主版本，再装 3.15 用于尝鲜。

> **“最新”与“最稳”不是一回事**
>
> 3.15 刚发布时，NumPy、PyTorch、数据库驱动等二进制包可能还没有对应 wheel，`pip install` 会尝试从源码编译，随后卡在编译器或系统库上。个人学习可以用最新版；生产环境优先选生态成熟、团队已验证的版本。

2.2 支持周期与 EOL

Python 采用“每年 10 月发布一个新次版本，约 5 年支持”的节奏：前两年 bugfix，之后只发安全修复。EOL 之后不再有安全补丁，继续使用会把已知漏洞留在机器上。

```
3.15  2026-10 发布   bugfix 至 2028-10   security 至 2031-10
3.14  2025-10 发布   bugfix 至 2027-10   security 至 2030-10
3.13  2024-10 发布   security 至 2029-10
3.12  2023-10 发布   security 至 2028-10
3.11  2022-10 发布   security 至 2027-10
3.10  2021-10 发布   已于 2026-10-01 EOL
```

检查一个项目的最低版本，不要只看代码里的语法，还要看依赖的 `Requires-Python`。可以在项目里执行：

```
$ python -m pip index versions 包名
$ python -m pip install "包名" --dry-run
```

`--dry-run` 会解析依赖但不真正安装，适合在升级 Python 前检查兼容性（需要较新的 pip）。

2.3 py、pyenv、uv、conda 怎么选

工具	主要职责	适合场景	不要用它做什么
py	Windows 启动器：按版本号调用已安装解释器	Windows 多版本共存、脚本指定版本	不负责下载 Python，不负责建环境
pyenv	下载/编译多个解释器，按目录切换	macOS/Linux 需要 3.10–3.15 频繁切换	不管理项目依赖；Windows 原生支持弱
uv	解释器下载、虚拟环境、依赖解析、锁文件	新项目、CI、追求速度	不是 conda 的完整替代（非 Python 依赖另说）
conda	解释器 + 二进制依赖 + 环境	数据科学、CUDA、GDAL、MKL	不要在 conda 环境里再套 venv
venv	标准库自带的项目级隔离	所有项目的基础做法	不能切换 Python 版本，不能管理非 Python 库

表 2-2 工具职责。最稳的组合是“一个解释器管理器 + 每个项目一个 venv”。

> **不要把所有工具都用上**
>
> 同一台机器同时用 pyenv、conda、uv、Homebrew 管同一个版本，最后一定会有至少一个 `python` 来源不明。选一个作为主入口，其他只在明确需要时使用。

2.4 目录、权限与命名

Python 的安装路径与项目路径会进入 `sys.path`、虚拟环境配置和 IDE 设置。路径里出现中文、空格、OneDrive 同步目录，短期可能没事，长期会在编译扩展、Jupyter kernel 或旧工具上出问题。

WINDOWS

推荐路径

`C:\Program Files\Python313`（全用户）或 `%LOCALAPPDATA%\Programs\Python\Python313`（当前用户）。项目放 `D:\code\项目名`，不要放桌面或 OneDrive。

macOS / LINUX

推荐路径

解释器交给 Homebrew/pyenv 管理；项目放 `~/code/项目名`。虚拟环境统一命名 `.venv`，放在项目根目录。

- **全用户 vs 当前用户**：个人电脑优先“当前用户”，不需要管理员权限；公司统一环境用“全用户”，路径固定、便于审计。
- **不要装到中文目录**：不是 Python 不支持，而是某些 C 扩展、构建工具和旧版 pip 对非 ASCII 路径处理不一致。
- **不要装进 OneDrive**：同步进程会锁文件，虚拟环境里成千上万个小文件会让同步变慢，甚至造成半同步状态。
- **统一环境名**：`.venv` 是事实标准，IDE、gitignore 模板和脚本都默认识别它。

2.5 一页决策卡

如果不想读完整章，按下面五行直接选。装完之后，仍然建议回到第 10 章做验收。

你的情况	解释器	项目环境	入口
Windows 个人电脑	3.13 + 3.15	`py -3.x -m venv .venv`	`py -3.13`
Windows 公司电脑	跟随 IT 批准版本	同上，环境放项目目录	`py -3.x`，不随意改 PATH
macOS 日常开发	Homebrew 一个主版本	`python3.x -m venv .venv`	`python3.x`
Linux 服务器	系统包 + `python3-venv`	项目 venv 或容器	`/usr/bin/python3` 不动
数据科学 / CUDA	conda 环境内解释器	`conda create -n ...`	`conda activate`

表 2-3 新电脑路线决策卡。选一个主入口，其他版本按需保留。

WINDOWS / OFFICIAL INSTALLER

03

Windows：官方安装器与 py 启动器

Windows 是最容易“装了很多版本却互相打架”的平台，也是最适合用 `py` 启动器统一入口的平台。这一章从下载页开始，逐项解释安装器选项，再给出一套可长期维护的多版本布局。

预计 30 分钟 | 需要管理员权限（可选） | Windows 10 / 11

下面以 Python 3.13 作为主版本、3.15 作为尝鲜版本。你可以把版本号换成 3.14 或团队要求的版本；安装器选项与命令结构不变。目标不是“把每个版本都装一遍”，而是保留 2–3 个有明确用途的版本。

3.1 安装器逐项选择

下载与校验

1. **确认架构**
   设置 → 系统 → 系统信息，查看“系统类型”。绝大多数是 64 位 x64；ARM 设备选择 ARM64 安装包。不要用 32 位安装包去“兼容”64 位系统。

2. **从 python.org 下载**
   打开 `python.org/downloads/windows/`，选择目标版本的 **Windows installer (64-bit)**。文件名形如 `python-3.13.7-amd64.exe`；3.15 的构建号会更新。

3. **核对来源与哈希**
   右键安装包 →“属性”→“数字签名”，确认签名有效；官网发布页通常提供 SHA-256，可用 `Get-FileHash .\python-3.13.7-amd64.exe -Algorithm SHA256` 对照。

4. **关闭正在使用 Python 的程序**
   IDE、Jupyter、终端、正在运行的脚本都可能占用 `python.exe`。升级或卸载前先关闭，否则会出现“装完了但文件没替换”。

第一屏：两个勾选项

安装器第一屏通常有两个关键选项：

- **Install launcher for all users (recommended)**：安装全局 `py` 启动器。建议勾选，这样所有账户都能用，且 `py -0p` 能看到已注册的解释器。
- **Add python.exe to PATH**：**建议不勾**，至少不要给每个版本都勾。多版本场景下，让 `py` 选择版本，比让 PATH 里堆满 `python.exe` 更可控。若你只装一个版本，可以勾选。

> **“Add to PATH”不是越多越好**
>
> 每个版本都勾选后，PATH 中会出现多个 `Python3xx\` 和 `Python3xx\Scripts\`。最后谁排在前面，`python` 和 `pip` 就听谁的。多版本环境请用 `py -3.x`，不要把每个版本都塞进 PATH。

Customize installation：Optional Features

点击 **Customize installation** 进入可选功能页。建议保留默认勾选，按需调整：

选项	建议	说明
Documentation	可选	本地文档；不装也能在线查。
pip	**必选**	包管理器。没有 pip，后续需要 `ensurepip` 补装。
tcl/tk and IDLE	可选	需要 IDLE 或 tkinter 图形界面时勾选。
Python test suite	不选	标准库测试套件，普通开发不需要。
py launcher	**必选**	多版本选择的核心；全用户安装时同时勾选“for all users”。

表 3-1 Optional Features。pip 与 py launcher 是多版本环境的基础。

Advanced Options：路径与注册

- **Install Python 3.x for all users**：装到 `C:\Program Files\Python3xx`，所有账户可用，需要管理员权限。个人电脑可勾；公司电脑按 IT 规范。
- **Associate files with Python**：让 `.py` 双击时用该版本运行。多版本机器上不建议勾，容易让双击行为与终端不一致。
- **Create shortcuts for installed applications**：创建开始菜单快捷方式，保留即可。
- **Add Python to environment variables**：等同于把该版本加入 PATH。**多版本场景不勾**；只装一个版本时可以勾。
- **Precompile standard library**：预编译标准库，首次启动更快，建议勾选。
- **Download debug symbols / debug binaries**：只有调试 CPython 或 C 扩展时才需要，普通开发不勾。

> **推荐组合（Windows 多版本）**
>
> 不勾 Add to PATH；勾选 py launcher（all users）；Customize → pip + py launcher；Advanced → 全用户安装（或当前用户）、预编译标准库；取消文件关联与环境变量。装完后用 `py -0p` 确认。

3.2 py 启动器与多版本

`py` 是 Windows 的 Python Launcher。它读取注册信息与 `py.ini`，按命令行里的版本号选择解释器；它不负责下载 Python，也不创建虚拟环境。

```
PS> py -0p
 -V:3.15 *        C:\Program Files\Python315\python.exe
 -V:3.13          C:\Program Files\Python313\python.exe
 -V:3.11          C:\Users\you\AppData\Local\Programs\Python\Python311\python.exe

PS> py -3.13 --version
PS> py -3.15 --version
PS> py --version          # 默认版本（带 * 的那个）
```

`-0p` 是“数字零 + p”，不是字母 O。它列出所有已注册解释器及路径；带 `*` 的是默认版本。若某个版本已安装但没出现在列表里，通常是安装时没注册 launcher，或安装的是便携版/嵌入式版。

设置默认版本

```
# 只对当前终端生效
PS> $env:PY_PYTHON = "3.13"

# 写入用户环境变量，重开终端后生效
PS> [Environment]::SetEnvironmentVariable("PY_PYTHON", "3.13", "User")

# 也可以写 py.ini（位置：%LOCALAPPDATA%\py.ini）
[defaults]
python=3.13
```

修改后重新打开终端，运行 `py --version` 确认。不要把 `PY_PYTHON` 与 PATH 里的 `python` 混为一谈：前者只影响 `py`，后者影响直接输入 `python` 时的解析。

> **py 不读取 pyenv 的 .python-version**
>
> 很多教程说“在项目里放一个 `.python-version`，`py` 就会自动切换”。Windows 官方 `py` 启动器并不保证读取这个文件；`.python-version` 是 pyenv、uv 等工具的约定。Windows 上要按项目固定版本，最稳的是把命令写进脚本：`py -3.13 -m venv .venv`，或使用 uv/pyenv-win。

用 py 创建虚拟环境

```
PS> cd D:\code\demo
PS> py -3.13 -m venv .venv
PS> .\.venv\Scripts\Activate.ps1
(.venv) PS> python --version
Python 3.13.7
(.venv) PS> python -m pip --version
```

关键点：`-m venv` 前面的解释器决定环境版本；进入环境后，`python` 与 `pip` 都指向 `.venv`，不再看全局 PATH。第 6 章会把这条链路拆开讲。

3.3 PATH 与 Microsoft Store 别名

Windows 查找命令的顺序由 PATH 决定。你可以在 PowerShell 里查看当前生效的命令与全部候选：

```
PS> Get-Command python -All | Select-Object Source
PS> where.exe python
PS> $env:Path -split ';'
```

如果最上面出现 `...\WindowsApps\python.exe`，而且文件大小是 0，那就是微软商店的“应用执行别名”。它不会真的运行 Python，而是弹出商店页面，非常容易被误判为“Python 坏了”。关闭方法：

1. **打开应用执行别名**
   设置 → 应用 → 高级应用设置 → 应用执行别名（不同 Windows 版本文字略有差异）。
2. **关闭 python.exe / python3.exe**
   把这两个开关关掉。以后 PATH 没配好时，终端会明确提示“找不到命令”，而不是静默打开商店。

检查项	正常结果	异常时
`where.exe python`	只出现你期望的入口，或没有输出	出现 WindowsApps、旧版本路径：调整 PATH 或关闭别名
`py -0p`	列出 2–3 个版本及真实路径	缺版本：重装并勾选 py launcher
`python -c "import sys; print(sys.executable)"`	输出当前解释器的绝对路径	输出与预期不符：PATH 或激活状态有问题

表 3-2 Windows 入口检查。先看路径，再看版本号。

3.4 多个版本的安装顺序

官方安装器可以并存，但建议按“从旧到新”或“从主到次”的顺序安装，并在每次安装后检查 `py -0p`。不要反复覆盖同一个目录。

```
C:\Program Files\Python313\       主版本，长期项目
C:\Program Files\Python315\       尝鲜版本，单独目录
C:\Users\you\AppData\Local\Programs\Python\Python311\    旧项目兼容
py                                C:\Windows\py.exe（全用户）
```

安装完 3.13 与 3.15 后：

```
PS> py -0p
PS> py -3.13 -m venv D:\code\project-a\.venv
PS> py -3.15 -m venv D:\code\project-b\.venv
```

两个项目各自绑定不同版本，互不影响。不要在项目 A 的 `.venv` 里再装项目 B 的依赖，也不要把两个环境合并。

> **安装顺序与默认版本**
>
> 最后安装的版本通常会成为 `py` 的默认版本（带 `*`），但这取决于安装器与已有配置，不要假设。装完后用 `PY_PYTHON` 或 `py.ini` 明确指定主版本。

3.5 验证、升级与卸载

验证

```
PS> py -0p
PS> py -3.13 --version
PS> py -3.13 -c "import sys; print(sys.executable)"
PS> py -3.13 -m pip --version
PS> py -3.13 -m venv $env:TEMP\py-check
PS> & $env:TEMP\py-check\Scripts\python.exe -c "import sys; print(sys.prefix)"
```

最后一行应输出临时环境目录，而不是全局 Python 目录。验证完可以删除 `$env:TEMP\py-check`，它只是测试环境。

升级

- **小版本升级**：下载同一次版本的新安装包（例如 3.13.7 → 3.13.8），覆盖安装到原目录。不要先卸载，否则可能丢失该版本的 site-packages 与脚本。
- **大版本升级**：新装一个目录（3.13 → 3.15），不要覆盖。然后为新项目创建新环境，旧项目继续用 3.13。
- **升级后**：重新打开终端，运行 `py -0p` 与 `py -3.13 --version`，确认路径与版本。

卸载

设置 → 应用 → 已安装的应用 → 找到对应版本的 Python → 卸载。官方卸载器会移除该版本的程序目录、开始菜单项和注册信息，但不会删除你的项目、虚拟环境或 pip 缓存。

> **不要直接删除安装目录**
>
> 直接删除 `C:\Program Files\Python313` 会留下注册表项与 py 启动器记录，导致 `py -0p` 仍显示一个不存在的路径。优先使用“应用和功能”卸载；便携版/嵌入式版除外。

MACOS / SYSTEM, PKG & HOMEBREW

04

macOS：系统 Python、官方包与 Homebrew

macOS 自带 Python，但它是给系统工具用的，不应该被你拿来装项目依赖。新电脑上要做的，是在“系统 Python 不动”的前提下，再加一个或多个可管理的版本，并让终端与 IDE 都指向正确入口。

预计 22 分钟 | Apple Silicon / Intel | zsh 默认

4.1 先看系统 Python

打开终端，先记录现状：

```
$ python3 --version
Python 3.9.6
$ which -a python3
/usr/bin/python3
$ xcode-select -p
/Library/Developer/CommandLineTools
$ uname -m
arm64        # Apple Silicon；Intel 显示 x86_64
```

`/usr/bin/python3` 属于 macOS 系统组件，版本通常较旧。不要 `sudo` 替换它，也不要把项目依赖装进它的 site-packages。系统更新可能覆盖或移除你做的修改，最终受伤的是系统工具。

> **Xcode CLT 与 Python 的关系**
>
> 安装 Xcode Command Line Tools 会提供 `/usr/bin/python3` 与编译工具，但它不是“给你开发用的最新 Python”。需要新版本时，用 python.org 安装包、Homebrew 或 pyenv。

4.2 python.org 安装包

这是最接近 Windows 官方安装器的方式：下载 macOS 安装包，按向导安装，得到一个独立的 Python.framework，不依赖 Homebrew。

1. **下载 universal2 安装包**

   打开 `python.org/downloads/macos/`，选择目标版本的 **macOS 64-bit universal2 installer**。universal2 同时支持 Apple Silicon 与 Intel。

2 **安装并验证路径**

双击 `.pkg`，按向导继续。安装后通常位于 `/Library/Frameworks/Python.framework/Versions/3.15`，命令在 `/usr/local/bin/python3.15`。

3 **确认 PATH 顺序**

运行 `which -a python3.15`。若 `/usr/local/bin` 不在 PATH 前面，按 4.3 节调整；不要直接改系统目录。

> **Gatekeeper 拦截**
>
> 若提示“无法验证开发者”，先确认下载来源是 python.org，再到“系统设置 → 隐私与安全性”点“仍要打开”。不要用 `spctl --master-disable` 全局关闭 Gatekeeper。

4.3 Homebrew 与 PATH

Homebrew 是 macOS 上最常用的包管理器。它把软件装在独立前缀：Apple Silicon 是 `/opt/homebrew`，Intel 是 `/usr/local`。安装 Python 时，优先使用带版本号的公式，避免多个版本争抢 `python3` 链接。

```
$ brew --version
$ brew install python@3.13
$ brew install python@3.15

# 查看版本化命令
$ which -a python3.13 python3.15
$ /opt/homebrew/bin/python3.13 --version
```

Homebrew 的 `python@3.x` 可能同时提供 `python3.x` 与一个未版本化的 `python3`。多个版本共存时，未版本化命令指向谁取决于链接状态，**不要依赖它**；脚本与项目里显式写 `python3.13` 或使用环境。

路径	来源	用途
`/opt/homebrew/bin`	Apple Silicon Homebrew	版本化 Python、pip、工具；应在 PATH 前部。
`/usr/local/bin`	Intel Homebrew / python.org	官方 pkg 的 `python3.15` 也在这里。
`/usr/bin`	macOS 系统	系统 Python，不要替换。

表 4-1　macOS 常见 Python 路径。终端调用 PATH 中第一个匹配项。

如果 `brew` 命令本身找不到，把 Homebrew 的 shellenv 加入 `~/.zshrc`：

```
# Apple Silicon
eval "$(/opt/homebrew/bin/brew shellenv)"

# Intel Mac
# eval "$(/usr/local/bin/brew shellenv)"
```

保存后执行 `source ~/.zshrc`，再运行 `hash -r` 与 `which -a python3.13`。

4.4 pyenv 多版本

如果你需要频繁在 3.10–3.15 之间切换，或者项目要求精确到补丁版本，pyenv 比 Homebrew 更合适。它把每个版本装在 `~/.pyenv/versions/`，通过 shim 目录切换。

```
# 安装编译依赖（Homebrew）
$ brew install openssl readline sqlite3 xz zlib tcl-tk

# 安装 pyenv
$ brew install pyenv

# 写入 shell 配置（zsh）
$ echo 'export PYENV_ROOT="$HOME/.pyenv"' >> ~/.zshrc
$ echo 'export PATH="$PYENV_ROOT/bin:$PATH"' >> ~/.zshrc
$ echo 'eval "$(pyenv init -)"' >> ~/.zshrc
$ exec zsh
```

```
$ pyenv install 3.13.7
$ pyenv install 3.15.2
$ pyenv versions
$ pyenv global 3.13.7
$ pyenv local 3.15.2     # 当前目录写 .python-version
$ python --version
```

`pyenv local` 会在当前目录写入 `.python-version`，进入该目录后自动切换；`pyenv global` 只改默认版本。不要把 `.python-version` 提交到所有项目，除非团队统一使用 pyenv 并希望固定版本。

> **macOS 上推荐组合**
>
> 日常主版本用 Homebrew 或 python.org；需要 3.10–3.15 多版本时用 pyenv；新项目用 `python3.x -m venv .venv`。不要同时用 Homebrew 的 `python3` 链接和 pyenv 的 shim 争抢同一个命令。

4.5 验证与卸载

```
$ python3.13 --version
$ python3.13 -c "import sys; print(sys.executable)"
$ python3.13 -m venv .venv
$ source .venv/bin/activate
(.venv) $ python -m pip --version
(.venv) $ deactivate
```

卸载按安装方式区分：Homebrew 用 `brew uninstall python@3.13`；官方 pkg 通常提供卸载脚本或需要手动移除 framework（先查安装器说明）；pyenv 用 `pyenv uninstall 3.13.7`。卸载不会删除项目里的 `.venv`，但该环境会失效，需要重建。

LINUX / PACKAGES, PPA & PYENV

05

Linux：系统包、deadsnakes 与 pyenv

Linux 的 Python 是系统的一部分，很多系统工具依赖 `/usr/bin/python3`。正确姿势不是“升级系统 Python”，而是保留系统版本，用版本化包、PPA 或 pyenv 提供额外的解释器。

预计 22 分钟 | 需要 sudo | Debian / Ubuntu / Fedora / Arch

5.1 包管理器与 venv 支持

先确认系统 Python 与 venv 模块。**很多“无法创建虚拟环境”的问题，不是 Python 没装，而是 `python3-venv` 没装。**

```
$ python3 --version
$ python3 -m venv --help > /dev/null && echo "venv OK"
```

发行版	安装命令	说明
Debian / Ubuntu	`sudo apt install python3 python3-venv python3-pip`	`python3-venv` 必须单独装，否则 `ensurepip` 不可用。
Fedora	`sudo dnf install python3 python3-pip`	venv 通常包含在 python3 包中。
RHEL / Rocky	`sudo dnf install python3 python3-pip`	系统版本较保守，新版本用 pyenv 或容器。
Arch	`sudo pacman -S python python-pip`	滚动更新，版本较新；不要随意替换系统 Python。
openSUSE	`sudo zypper install python3 python3-pip`	venv 在 python3 包内。
Alpine	`sudo apk add python3 py3-pip`	容器常用；需补 `build-base` 才能编译 C 扩展。

表 5-1 常见发行版的 Python 与 venv 安装。包名可能随版本变化，以仓库为准。

5.2 Ubuntu deadsnakes PPA

Ubuntu 官方仓库的 Python 版本通常比 python.org 旧。需要 3.13/3.14/3.15 时，可以使用 deadsnakes PPA。它是社区维护的第三方仓库，生产环境引入前确认公司政策。

```
$ sudo apt update
$ sudo apt install software-properties-common
$ sudo add-apt-repository ppa:deadsnakes/ppa
$ sudo apt update
$ sudo apt install python3.13 python3.13-venv python3.13-dev
$ python3.13 --version
```

`python3.13-dev` 只在编译 C 扩展时需要；`python3.13-venv` 一定要装，否则创建环境会报 `ensurepip is not available`。安装后命令是版本化的 `python3.13`，不要试图用它替换 `python3`。

> **不要用 update-alternatives 改系统 python3**
>
> 把 `/usr/bin/python3` 指向 3.13 可能让 apt、发行版工具和桌面组件失效。需要新版本时，显式调用 `python3.13`；需要默认入口时，用 shell 别名、pyenv 或项目脚本，不要动系统链接。

5.3 pyenv 编译安装

pyenv 适合需要多个精确版本、又不想依赖发行版仓库的场景。它从源码编译 Python，所以要先装编译依赖。

```
# Debian / Ubuntu 编译依赖
$ sudo apt install -y make build-essential libssl-dev zlib1g-dev \
  libbz2-dev libreadline-dev libsqlite3-dev wget curl llvm \
  libncurses-dev xz-utils tk-dev libxml2-dev libxmlsec1-dev libffi-dev liblzma-dev

# 安装 pyenv
$ curl -fsSL https://pyenv.run | bash
```

按安装脚本提示，把下面三行加入 `~/.bashrc` 或 `~/.zshrc`：

```
$ export PYENV_ROOT="$HOME/.pyenv"
$ [[ -d $PYENV_ROOT/bin ]] && export PATH="$PYENV_ROOT/bin:$PATH"
$ eval "$(pyenv init -)"
```

```
$ pyenv install 3.13.7
$ pyenv install 3.15.2
$ pyenv versions
$ pyenv global 3.13.7
$ pyenv local 3.15.2
$ python --version
```

如果编译失败，先看日志里的缺失头文件（常见是 `bz2`、`sqlite3`、`ssl`），补齐依赖后执行 `pyenv install --force 3.13.7`。不要用 `--enable-shared` 之外的随机参数去“优化”，除非你知道自己在做什么。

5.4 WSL、容器与最小化环境

WSL2

在 WSL 的 Linux 发行版里单独安装 Python，不要把 Windows 的 `python.exe` 当成 WSL 的默认解释器。项目放在 Linux 文件系统（如 `~/code`），不要放在 `/mnt/c`，否则 I/O 慢且权限/换行符容易出问题。

```
$ sudo apt update
$ sudo apt install python3 python3-venv python3-pip
$ cd ~/code/demo
$ python3 -m venv .venv
$ source .venv/bin/activate
```

Docker / CI

容器里通常不需要 venv，因为镜像本身就是隔离边界；但如果要与开发环境保持一致，也可以在镜像里创建。最小化镜像要补 `ca-certificates` 与编译依赖。

```
# 直接使用官方 Python 镜像
FROM python:3.13-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

5.5 不要动系统 python3

现代 Debian/Ubuntu 会标记系统 Python 为 “externally managed”（PEP 668）。直接 `pip install` 到系统环境会被拒绝，并提示使用 venv 或 `--break-system-packages`。

> **不要用 --break-system-packages 图省事**
>
> 它会让 pip 覆盖发行版管理的包，下一次系统更新可能冲突，甚至影响 apt、桌面环境或其他系统工具。正确做法是项目 venv；命令行工具用 `pipx`；需要新解释器用 deadsnakes、pyenv 或容器。

验证系统解释器与项目解释器互不影响：

```
$ /usr/bin/python3 --version
$ which python3
$ python3 -m venv .venv
$ source .venv/bin/activate
(.venv) $ which python
(.venv) $ python -m pip --version
```

进入环境后，`which python` 应指向项目下的 `.venv/bin/python`，而不是 `/usr/bin/python3`。第 6 章会讲这条链路为什么可靠。

5.6 发行版验收卡

Linux 上的“装好”不是 `python3 --version` 有输出，而是系统解释器、版本化解释器、venv 与 pip 四者边界清楚。用下面五项快速验收：

检查	命令	期望
系统解释器	`/usr/bin/python3 --version`	正常返回；不要替换。
版本化解释器	`python3.13 --version`	返回目标版本。
venv 模块	`python3.13 -m venv --help`	输出帮助，不报 ensurepip 错误。
项目环境	`python3.13 -m venv .venv`	目录生成，激活后 `sys.prefix` 指向项目。
pip 归属	`python -m pip --version`	路径在 `.venv` 内。

表 5-2 Linux 环境验收。第 3 项失败时先装 `python3.x-venv`。

> **系统包与项目包各归其位**
>
> 系统包由 apt/dnf 管理，项目包由 venv 管理。两者不要互相覆盖；需要命令行工具时用 `pipx`，需要新解释器时用版本化包或 pyenv。

VENV / PROJECT ISOLATION

06

venv：项目级隔离的核心

`venv` 是 Python 标准库自带的虚拟环境工具。它不下载新版本，也不复制整个 Python；它创建一个小目录，让 `python`、`pip` 和 `site-packages` 在这个目录里有一套独立的入口。

预计 20 分钟 跨平台 每个项目必做

6.1 venv 到底做了什么

在项目根目录执行 `python -m venv .venv` 后，目录里会出现这些关键内容（Windows 与 macOS/Linux 的目录名略有不同）：

```
.venv/
├── pyvenv.cfg# 记录基础解释器路径、版本、是否包含系统包
├── Scripts/# Windows：python.exe、pip.exe、activate*
│   ├── python.exe
│   ├── pip.exe
│   ├── Activate.ps1
│   └── activate.bat
└── Lib/site-packages/# 当前环境安装的第三方包# macOS / Linux 对应：bin/ 与 lib/python3.x/site-packages/
```

核心不是“复制了一个 Python”，而是创建了一个带 `pyvenv.cfg` 的目录，并让其中的解释器在启动时把 `sys.prefix` 指到这个目录。这样 `site-packages` 就与全局解释器分开。

对象	venv 内	venv 外
`sys.prefix`	项目下的 `.venv`	Python 安装目录
`site-packages`	`.venv/Lib/site-packages`	全局 Python 的 site-packages
标准库	仍来自基础解释器	同一份标准库
Python 版本	由创建时的基础解释器决定	由安装的版本决定

表 6-1 venv 隔离的是包与入口，不是标准库，也不是操作系统。

> **venv 不是安全沙箱**
>
> 它只隔离 Python 包，不限制文件系统、网络或进程权限。不要用它运行来路不明的代码，也不要把“装了 venv”当成安全边界。

6.2 创建环境的正确命令

统一用“目标解释器 + `-m venv` + 目录名”的形式。下面三种写法等价，区别只是你用哪个入口找到目标解释器：

```
# Windows：用 py 启动器指定版本
PS> cd D:\code\demo
PS> py -3.13 -m venv .venv

# Windows：已知 python.exe 的绝对路径
PS> "C:\Program Files\Python313\python.exe" -m venv .venv

# macOS / Linux：使用版本化命令
$ cd ~/code/demo
$ python3.13 -m venv .venv
```

建议加上 `--upgrade-deps`，让 venv 在创建时顺便升级 pip、setuptools 与 wheel（Python 3.9+ 支持）：

```
$ python3.13 -m venv --upgrade-deps .venv
```

不要用 `python -m venv` 而不确认 `python` 是谁。创建前先运行 `python -c "import sys; print(sys.executable, sys.version)"`，确保它是你想要的版本。

> **不要用 virtualenv 替代 venv 作为默认**
>
> `virtualenv` 是第三方工具，速度快、支持旧版本，但新项目没有必要引入额外依赖。标准库 `venv` 足够；只有在需要兼容 Python 2/3 或特殊加速时才考虑 virtualenv。

6.3 激活与不激活

“激活”本质上是修改当前 shell 的 `PATH`，让 `python`、`pip` 先指向 `.venv`。它不是必需的：你完全可以用绝对路径直接调用环境里的解释器。

Shell	激活命令	退出
PowerShell	`.\.venv\Scripts\Activate.ps1`	`deactivate`
CMD	`.\.venv\Scripts\activate.bat`	`deactivate`
Git Bash	`source .venv/Scripts/activate`	`deactivate`
macOS / Linux	`source .venv/bin/activate`	`deactivate`
不激活	`.venv\Scripts\python.exe -m pip install ...` 或 `.venv/bin/python -m pip ...`	—

表 6-2 各 shell 的激活方式。激活后提示符通常会出现 `(.venv)`。

PowerShell 执行策略

如果 `Activate.ps1` 报“无法加载文件，因为在此系统上禁止运行脚本”，只为当前用户放开签名脚本即可：

```
PS> Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
PS> .\.venv\Scripts\Activate.ps1
```

不要用 `Bypass` 或 `Unrestricted` 作为长期设置。若公司有组策略，按 IT 规定申请签名或改用 CMD 的 `activate.bat`。

不激活也能工作

```
# 明确使用环境里的解释器，适合脚本与 CI
$ .venv/bin/python -m pip install -r requirements.txt
$ .venv/bin/python app.py

# Windows
PS> .\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

> **激活不是“进入环境”的开关**
>
> 激活只改变当前 shell 的 PATH。IDE、Jupyter、计划任务不会共享这个状态。自动化脚本应使用解释器绝对路径，而不是依赖“先激活”。

6.4 解释器绑定与重建

虚拟环境在创建时绑定基础解释器，之后不能通过 `activate` 切换 Python 版本。想换版本，正确做法是删除旧环境、用新解释器重建：

```
PS> deactivate
PS> Remove-Item -Recurse -Force .venv
PS> py -3.15 -m venv .venv
PS> .\.venv\Scripts\Activate.ps1
(.venv) PS> python -m pip install -r requirements.txt
```

macOS / Linux 把删除命令换成 `rm -rf .venv`，创建命令换成 `python3.15 -m venv .venv`。如果依赖已经稳定，先导出锁定文件再删除，避免重建后版本漂移。

> **不要移动或重命名 venv**
>
> `pyvenv.cfg` 与激活脚本里记录了绝对路径或相对基础路径。把 `.venv` 从 `D:\code\a` 移到 `D:\code\b`，或复制到另一台电脑，常见结果是 `python` 找不到基础解释器、pip 指向旧路径。虚拟环境应“就地重建”，不要跨机器复制。

查看绑定关系：

```
(.venv) $ python -c "import sys; print(sys.executable); print(sys.prefix); print(sys.base_prefix)"
(.venv) $ cat .venv/pyvenv.cfg
```

`sys.prefix` 是当前环境，`sys.base_prefix` 是基础解释器。两者相同说明你没有在虚拟环境里；不同才说明 venv 生效。

6.5 常见选项与隔离边界

选项	用途与注意
`--system-site-packages`	让环境也能看到全局 site-packages。方便复用系统包，但会破坏隔离，依赖冲突更难排查；默认不要用。
`--copies`	复制解释器而不是创建符号链接。某些文件系统/Windows 环境下更稳，但占用更多空间。
`--prompt 名称`	自定义激活提示符，适合同时开多个环境时区分。
`--without-pip`	不安装 pip；只有你自己管理包时才用。普通项目不要。
`--upgrade-deps`	创建时升级 pip/setuptools/wheel，推荐。

表 6-3 venv 常用选项。默认配置已经适合大多数项目。

一个常见误解是“venv 会让项目变小”。实际上每个环境都会安装一份依赖，磁盘占用可能不小；它换来的是隔离与可复现。定期清理不再使用的环境，而不是把所有包都装进全局。

- ☐ **每个项目**一个 `.venv`，不跨项目复用。
- ☐ **不提交** `.venv`；只提交依赖声明与锁定文件。
- ☐ **不复制**虚拟环境到另一台机器；在新机器重建。
- ☐ **不混用**conda 环境与 venv；二选一。

PIP / DEPENDENCIES

07

pip、依赖与可复现

venv 解决“装在哪里”，pip 解决“装什么、装哪个版本”。新电脑上最容易复现失败的地方，不是 Python 本身，而是依赖没有锁定、pip 指向错环境、镜像与证书配置不一致。

预计 25 分钟 | pip 25+ | requirements / pyproject

7.1 永远用 python -m pip

`pip` 是脚本，`python -m pip` 是“用当前解释器运行 pip 模块”。后者能保证 pip 与解释器是同一个环境，不受 PATH 中其他 `pip.exe` 影响。

```
# 推荐
(.venv) $ python -m pip install requests

# 不推荐：pip 可能来自另一个环境
(.venv) $ pip install requests
```

验证 pip 属于谁：

```
(.venv) $ python -c "import sys; print(sys.executable)"
(.venv) $ python -m pip --version
pip 25.2 from .../.venv/lib/python3.13/site-packages/pip (python 3.13)
```

如果 `python -m pip --version` 的路径不在 `.venv` 下，说明当前 shell 没有激活或 PATH 有问题；此时 `pip install` 极可能装到全局。

> **为什么很多教程仍写 pip**
>
> 在只有一个 Python、且已激活环境时，`pip` 与 `python -m pip` 通常等价。但多版本机器上，这个假设经常不成立。把 `python -m pip` 当成默认写法，能省掉大量“包装了但找不到”的排查。

7.2 升级与常用命令

```
# 升级 pip 自身
(.venv) $ python -m pip install --upgrade pip

# 一次升级构建三件套
(.venv) $ python -m pip install --upgrade pip setuptools wheel

# 安装 / 卸载 / 查看
(.venv) $ python -m pip install requests
(.venv) $ python -m pip uninstall requests
(.venv) $ python -m pip list
(.venv) $ python -m pip show requests
(.venv) $ python -m pip check
```

`pip check` 检查已安装包之间的依赖是否冲突。它不会修复冲突，但能快速告诉你“某个包要求另一个包的版本范围不满足”。

命令	用途
`python -m pip list --outdated`	列出可升级的包；升级前先看变更日志。
`python -m pip install -r requirements.txt`	按文件安装依赖。
`python -m pip install -e ".[dev]"`	可编辑安装当前项目，并带上 dev 可选依赖。
`python -m pip cache dir` / `cache purge`	查看/清理 pip 下载缓存，释放磁盘。
`python -m pip download -r requirements.txt -d wheels`	离线环境预下载 wheel。

表 7-1 pip 常用命令。所有命令都在已激活的项目环境中执行。

7.3 requirements 的三种写法

`requirements.txt` 简单直接，但“怎么写”决定了可复现性。下面三种写法适用于不同场景：

写法 A：直接依赖（适合库或简单项目）

```
# requirements.txt
requests>=2.32,<3
numpy>=2.1,<3
pytest>=8,<9
```

只写你直接使用的包，让 pip 解析传递依赖。升级时会拿到兼容范围内的新版本，适合持续维护的项目。

写法 B：完全冻结（适合应用部署）

```
(.venv) $ python -m pip freeze > requirements.lock.txt
```

Windows PowerShell 5.1 注意： `>` 默认写 UTF-16，部分工具读取时会出问题。改用 `python -m pip freeze | Set-Content -Encoding utf8 requirements.lock.txt`，或使用 PowerShell 7。

输出所有包及精确版本，可复现性最高。但 `freeze` 会把本地可编辑安装、路径依赖、平台特定包一起写进去，跨平台不一定可用；库项目不要把 freeze 结果当发布依赖。

写法 C：分层与约束（适合团队）

```
# requirements.in（直接依赖）
requests>=2.32
pytest>=8

# 生成锁定文件（需要 pip-tools）
$ pip-compile requirements.in -o requirements.txt
$ pip-sync requirements.txt
```

`pip-tools` 把“声明”和“锁定”分开，既保留可读性，又能精确复现。对中小项目，这是比手写 freeze 更稳的做法。

> **不要提交带本机路径的 freeze**
>
> 如果 `pip freeze` 出现 `-e git+file:///D:/...` 或本地绝对路径，说明环境里装了可编辑项目。这样的文件换机器必然失败；应改为从版本库或包索引安装。

7.4 pyproject.toml 与可编辑安装

现代 Python 项目用 `pyproject.toml` 声明元数据、依赖与构建后端。最小示例：

```
# pyproject.toml
[project]
name = "demo"
version = "0.1.0"
requires-python = ">=3.13,<3.16"
dependencies = [
    "requests>=2.32,<3",
]

[project.optional-dependencies]
dev = ["pytest>=8", "ruff>=0.9"]

[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"
```

安装项目本身与开发依赖：

```
(.venv) $ python -m pip install -e ".[dev]"
```

`-e` 是可编辑安装：源码改动立即生效，适合开发。若项目是库，发布时用 `python -m build` 生成 wheel/sdist，不要在 requirements 里写本地路径。

文件	作用	是否提交
`pyproject.toml`	项目元数据、依赖、构建配置	提交
`requirements.txt`	应用部署的精确依赖	提交（按团队约定）
`uv.lock` / `poetry.lock`	锁定完整解析结果	提交（应用项目）
`.venv/`	本机环境	**不提交**

表 7-2 项目依赖文件。锁文件是否提交取决于项目类型与团队规范。

7.5 镜像、代理与证书

国内网络下，PyPI 直连可能很慢。可以配置镜像，但要清楚它的信任边界：镜像能看到你请求的包名，企业私有包不应走公共镜像。

```
# 临时使用镜像
(.venv) $ python -m pip install -i https://pypi.tuna.tsinghua.edu.cn/simple requests

# 写入用户配置
$ python -m pip config set global.index-url https://pypi.tuna.tsinghua.edu.cn/simple
$ python -m pip config list
```

推荐把镜像写在用户级配置，而不是项目文件；项目需要私有索引时，用 `--extra-index-url` 并明确优先级。**不要**为了省事关掉 TLS 校验。

```
# 企业代理
$ python -m pip install --proxy http://127.0.0.1:7890 requests
$ export HTTPS_PROXY=http://127.0.0.1:7890

# 企业自签 CA（路径示例）
$ python -m pip config set global.cert /path/to/company-ca.pem
```

如果出现 `SSLError` 或 `certificate verify failed`，先确认系统时间、代理和公司根证书。不要用 `--trusted-host` 长期绕过校验；它会让中间人攻击变得容易。

7.6 依赖冲突与 pip check

pip 的解析器会尽量找到一组满足所有约束的版本，但遇到“A 要求 numpy<2，B 要求 numpy>=2”这类硬冲突时，只能报错。处理顺序：

1. **先看完整报错**
   找到是哪个包、哪个版本范围冲突，不要直接 `--force-reinstall`。
2. **确认 Python 版本**
   某些包的新版本只支持 3.13+，你在 3.11 上永远装不到。
3. **拆环境**
   如果两个工具无法共享依赖，给它们各自建环境，而不是强行合并。

```
(.venv) $ python -m pip check
(.venv) $ python -m pip install "numpy<2" "pandas>=2.2" --dry-run
```

`--dry-run` 只解析不安装，适合在改动环境前确认版本组合。若需要可视化依赖树，可安装 `pipdeptree`，但它本身也是第三方包，注意版本。

WORKFLOW / PROJECT & IDE

08

项目工作流与 IDE

环境配置好之后，真正决定体验的是“每天怎么用”：先激活还是直接用绝对路径？VS Code 选哪个解释器？Jupyter 为什么看不到新环境？这一章把项目结构、编辑器和启动脚本串起来。

预计 20 分钟 | VS Code / PyCharm | Jupyter

8.1 一个项目一个环境

推荐的最小循环：进入项目 → 激活环境 → 安装/运行 → 退出。不要把“激活”当成全局状态；每个终端窗口独立。

```
PS> cd D:\code\demo
PS> .\.venv\Scripts\Activate.ps1
(.venv) PS> python -m pip install -r requirements.txt
(.venv) PS> python app.py
(.venv) PS> deactivate
```

如果你经常忘记激活，使用“解释器绝对路径”更稳：

```
PS> .\.venv\Scripts\python.exe app.py
PS> .\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

这两种方式没有优劣，关键是团队统一。脚本、CI、计划任务应使用绝对路径；交互式开发可以激活。

8.2 目录结构与 .gitignore

```
demo/
├── .venv/# 本机环境，不提交
├── src/demo/
│   ├── __init__.py
│   └── main.py
├── tests/
├── pyproject.toml
├── requirements.txt
├── .env.example        # 变量名模板，不含真实密钥
├── .gitignore
└── README.md
```

```
# .gitignore
.venv/
__pycache__/
*.py[cod]
.pytest_cache/
.ruff_cache/
.mypy_cache/
build/
dist/
*.egg-info/
.env
.env.*
!.env.example
```

不要提交 `.venv`：它包含绝对路径、平台相关二进制和大量小文件。也不要提交 `.env`；只提交 `.env.example`，让新机器知道需要哪些变量。

> **已经提交了 .venv 怎么办**
>
> 先把它加入 `.gitignore`，再执行 `git rm -r --cached .venv`，提交“停止跟踪”。历史里仍会留下大文件；如果仓库已经很大，再评估是否需要用 `git filter-repo` 清理历史。

8.3 VS Code、PyCharm 与 Jupyter

VS Code

1. 打开项目文件夹，按 `Ctrl+Shift+P`（macOS `Cmd+Shift+P`）。
2. 执行 **Python: Select Interpreter**，选择项目下的 `.venv/Scripts/python.exe` 或 `.venv/bin/python`。
3. 新建终端，确认提示符前有 `(.venv)`；VS Code 通常会自动激活选中的环境。
4. 如果终端没有激活，执行 `Python: Create Terminal`，或手动运行激活命令。

可以在项目里保存推荐设置，但不要写死本机绝对路径：

```
# .vscode/settings.json
{
  "python.defaultInterpreterPath": ".venv/Scripts/python.exe",
  "python.terminal.activateEnvironment": true,
  "python.testing.pytestEnabled": true
}
```

Windows 与 macOS/Linux 的路径不同，团队可以提交 `.vscode/settings.json` 的通用部分，把解释器路径留给本机设置。

PyCharm

Settings → Project → Python Interpreter → Add Interpreter → Existing，选择项目 `.venv` 里的解释器。PyCharm 会把它标记为项目解释器，运行配置与终端都应使用它。若终端仍是系统 Python，检查 Settings →

Tools → Terminal 的“Activate virtualenv”选项。

Jupyter

Jupyter 不会自动发现所有 venv。需要把当前环境注册为 kernel：

```
(.venv) $ python -m pip install ipykernel
(.venv) $ python -m ipykernel install --user --name demo-3.13 --display-name "Python 3.13 (demo)"

# 查看 / 删除 kernel
$ jupyter kernelspec list
$ jupyter kernelspec remove demo-3.13
```

注册后，Notebook 右上角选择对应 kernel。若代码里仍然找不到包，先确认 kernel 的 `sys.executable` 是否指向 `.venv`，而不是只看 Notebook 的显示名称。

8.4 .env 与配置

把密钥、数据库地址等放在环境变量里，不要写进代码或提交到仓库。开发环境可以用 `.env` 文件配合 `python-dotenv`；生产环境用平台注入的环境变量。

```
# .env（不提交）
DATABASE_URL=postgresql://localhost/demo
API_KEY=replace-me

# .env.example（提交）
DATABASE_URL=
API_KEY=
```

```
# 读取（需要 python-dotenv）
from dotenv import load_dotenv
import os

load_dotenv()
db = os.environ["DATABASE_URL"]
```

`.env` 只适合本地开发，不是密钥管理系统。团队共享的密钥应放在 1Password、Vault 或云平台的 Secret Manager 中，并设置轮换策略。

8.5 初始化脚本

把重复步骤写成脚本，能避免“每台机器都靠记忆”。脚本至少要做：检查 Python 版本 → 创建 `.venv` → 升级 pip → 安装依赖 → 打印解释器路径。完整模板见附录 B。

```
# 手动版（Windows PowerShell）
PS> py -3.13 -m venv --upgrade-deps .venv
PS> .\.venv\Scripts\python.exe -m pip install -r requirements.txt
PS> .\.venv\Scripts\python.exe -c "import sys; print(sys.executable)"
```

脚本里不要写死 `C:\Users\你的名字` 或 `/home/你的名字`；用脚本所在目录或项目根目录，保证换电脑、换用户名后仍可运行。

MULTI-VERSION / TESTING

09

多版本并用：测试、切换与冲突

多版本不是目的，能针对不同项目、不同 CI 目标稳定运行才是。这一章讲三件事：如何用同一份代码测多个版本，如何用 uv/conda 管理不同工作流，以及最常见的五种冲突如何拆开。

预计 22 分钟 | tox / nox | uv / conda

9.1 同一项目测多个版本

库项目通常需要支持多个 Python 版本。不要手动建五个环境再逐个跑测试；用矩阵配置让工具自动创建、安装、执行。

版本	角色	在本机怎么获得
3.11	最低支持版本	官方安装器 / pyenv / deadsnakes
3.13	主力开发版本	官方安装器 / Homebrew / 系统包
3.15	最新版本	官方安装器 / pyenv / uv

表 9-1 矩阵示例。实际范围由 `requires-python` 与依赖支持情况决定。

项目元数据里写清支持范围：

```
[project]
requires-python = ">=3.11,<3.16"
```

9.2 tox 与 nox

`tox` 用声明式配置创建隔离环境；`nox` 用 Python 脚本描述会话。两者都能覆盖多版本矩阵。下面是 tox 的最小示例：

```
# tox.ini
[tox]
env_list = py311, py313, py315

[testenv]
deps = pytest
commands = pytest -q
```

```
$ python -m pip install tox
$ tox
$ tox -e py313
```

tox 会为每个环境创建独立目录（通常在 `.tox/`），并调用对应的 `python3.11`、`python3.13`、`python3.15`。如果某个版本没有安装，tox 会跳过或报错；先确保解释器在 PATH 或通过 `basepython` 指定。

nox 示例：

```
# noxfile.py
import nox

@nox.session(python=["3.11", "3.13", "3.15"])
def tests(session):
    session.install("-r", "requirements.txt")
    session.run("pytest", "-q")
```

9.3 uv 工作流

uv 把“下载解释器、建环境、装依赖、锁版本”合成一套快速流程。适合新项目与 CI，但团队要统一使用，否则锁文件会打架。

```
# 安装 uv（Windows）
PS> winget install --id=astral-sh.uv -e

# 安装解释器
PS> uv python install 3.13 3.15
PS> uv python list

# 新项目
PS> uv init demo
PS> cd demo
PS> uv python pin 3.13
PS> uv add requests
PS> uv run python main.py
```

uv 项目通常包含 `pyproject.toml` 与 `uv.lock`。新机器上执行 `uv sync` 即可复现环境，不需要手动 `venv` + `pip install`。如果团队不用 uv，不要提交 `uv.lock` 后又让别人用 pip 猜版本。

> **uv 与 venv 的关系**
>
> uv 默认仍会在项目里创建 `.venv`，只是创建与安装更快。它没有把虚拟环境“消灭”，只是把流程自动化了。理解 venv 仍然是必要的。

9.4 conda 工作流

conda 适合需要非 Python 依赖的场景（CUDA、MKL、GDAL、编译好的科学计算库）。它自己管理解释器与环境，不要再套 venv。

```
# 创建指定版本的环境
$ conda create -n py313 python=3.13 numpy pandas
$ conda activate py313
$ python --version
$ conda list

# 安装包：优先 conda，找不到再 pip
$ conda install scipy
$ python -m pip install some-package

# 导出 / 删除
$ conda env export --from-history > environment.yml
$ conda deactivate
$ conda env remove -n py313
```

`base` 只作为启动器，不要往里装项目依赖。新项目用独立环境；升级 base 可能破坏已有的科学计算栈。

> **不要叠加 conda 与 venv**
>
> 在已激活的 conda 环境里再 `python -m venv .venv`，会得到一个“基于 conda 解释器的 venv”。它能用，但排查时 `sys.base_prefix`、PATH、conda 包与 pip 包交织在一起。选一层隔离即可。

9.5 五个真实冲突案例

01 **“我装了 3.13，为什么 `python` 还是 3.9？”**

PATH 里 3.9 在前，或 3.13 没注册到 `py`。用 `py -0p` 与 `where python` 分别看；不要继续装第 4 个版本。

02 **“conda activate 后 pip 装到了别处。”**

先看 `python -m pip --version`。如果路径不在 conda 环境内，说明 shell 初始化没生效或另一个 pip 在 PATH 前。用 `python -m pip` 而不是裸 `pip`。

03 **“pyenv 切了版本，VS Code 还是旧版本。”**

VS Code 的解释器是独立设置，不跟随 shell 的 pyenv shim。重新执行 Python: Select Interpreter，选中 `~/.pyenv/versions/3.13.7/bin/python`。

04 **“同一个项目，两个依赖永远冲突。”**

如果它们确实要求互斥版本，拆成两个环境或两个服务，而不是强行 `--force-reinstall`。用 `pip check` 确认冲突链。

05 **“M1/M2 Mac 上装不上某个包。”**

确认包是否有 arm64 wheel；没有就考虑 Rosetta 终端、conda-forge 或从源码编译。不要用 `--no-binary` 盲目强制源码构建。

9.6 多版本工作流选择

场景	推荐工具	原因
只维护一个应用	一个主版本 + venv	依赖稳定，入口越少越好。
库项目，需兼容多版本	tox / nox + pyenv	自动创建矩阵环境，测试结果可复现。
数据科学 / CUDA	conda	非 Python 依赖由 conda 解决，避免源码编译。
CI / 新项目	uv	下载解释器与锁依赖快，适合缓存与自动化。

表 9-2 多版本工作流。可以组合，但同一项目只保留一个“安装真相”。

选择原则

先确定“谁负责解释器”，再确定“谁负责项目依赖”。解释器层选 py、pyenv、uv 或 conda 中的一个；项目层统一用 venv 或所选工具自己的环境。两层都换工具，排查成本会翻倍。

- [ ] **命令可复现。**把测试矩阵写进 CI，不依赖某台机器的默认版本。
- [ ] **环境不叠加。**conda 环境里不再建 venv；venv 里不再装另一个 Python。
- [ ] **锁文件唯一。**同一项目不要同时维护 uv.lock 与 requirements.txt 两套真相。
- [ ] **失败可定位。**每个环境都能打印 `sys.executable` 与 pip 来源。

VERIFICATION / DIAGNOSTICS

10

验收与诊断

安装完成不等于环境可用。真正的验收要覆盖四层：解释器能找到、版本正确、venv 绑定正确、依赖能装能跑。下面十二项检查可以在十分钟内跑完，适合新电脑交付与故障复现。

预计 15 分钟 跨平台 建议保存为脚本

10.1 十二项验收清单

#	检查项	命令	通过标准
1	解释器可见	`py -0p` / `which -a python3`	列出目标版本与真实路径。
2	版本正确	`py -3.13 --version` / `python3.13 --version`	返回预期次版本。
3	入口可解释	`python -c "import sys; print(sys.executable)"`	输出与预期解释器一致。
4	pip 归属正确	`python -m pip --version`	路径在目标解释器或 venv 内。
5	venv 可创建	`py -3.13 -m venv .venv`	目录生成，退出码 0。
6	venv 已激活	`.\.venv\Scripts\Activate.ps1`	提示符出现 `(.venv)`。
7	环境内解释器	`python -c "import sys; print(sys.prefix); print(sys.base_prefix)"`	两者不同，`prefix` 指向项目 `.venv`。
8	依赖可安装	`python -m pip install requests`	安装成功且不提示权限错误。
9	依赖可导入	`python -c "import requests; print(requests.__version__)"`	输出版本号。
10	依赖一致	`python -m pip check`	输出 “No broken requirements found.”
11	脚本可运行	`python app.py`	项目入口正常执行。
12	退出可复原	`deactivate` 后再查 `python`	回到系统解释器，不残留环境变量。

表 10-1 十二项验收。第 8–11 项在一个临时项目中执行，避免污染真实项目。

10.2 诊断命令与正常输出

验收失败时，先收集事实，不要重装。下面几条命令能回答“是谁、在哪、为什么”三个问题。

命令	回答什么	怎么看
`py -0p` / `py --list-paths`	Windows 注册了哪些版本	看路径是否存在、带 `*` 的默认版本。
`where.exe python` / `which -a python3`	PATH 中所有候选	第一条就是当前入口；留意 WindowsApps、旧版本。
`python -c "import sys; print(sys.executable); print(sys.path)"`	当前解释器与模块搜索路径	`sys.path` 是否包含意外目录。
`python -m site`	site-packages 与用户目录	确认没有把包装进用户全局目录。
`python -m pip --version`	pip 属于哪个环境	路径应包含 `.venv` 或目标解释器目录。
`python -m pip config list -v`	pip 配置来自哪里	检查 index-url、proxy、cert 是否符合预期。
`python -m pip check`	依赖是否冲突	有输出就按提示定位，不要直接忽略。

表 10-2 诊断命令。把输出保存到文件，比截图更容易比对。

```
# 一条命令生成环境快照（macOS / Linux / Git Bash）
$ { python --version; which -a python; python -c "import sys; print(sys.executable); print(sys.prefix); print(sys.base_prefix)"; python -m pip --version; python -m pip check; } > env-report.txt 2>&1
```

10.3 网络、镜像与证书

pip 安装失败不一定是包不存在。先区分 DNS、代理、TLS 与索引四层：

PowerShell

```
PS> Resolve-DnsName pypi.org
PS> Test-NetConnection pypi.org -Port 443
PS> python -m pip download requests -d $env:TEMP\pip-test -v
```

macOS / Linux

```
$ curl -I https://pypi.org/simple/
$ python -m pip download requests -d /tmp/pip-test -v
```

`-v` 会显示实际使用的 index、代理与证书路径。若公司使用自签 CA，优先配置 `global.cert` 或系统证书库；不要用 `--trusted-host` 永久绕过。

现象	原因	处理
`Could not find a version`	索引没有该包/该版本，或 Python 版本不兼容	检查包名、Python 版本、wheel 标签。
`SSLError` / 证书错误	代理拦截、自签根证书、系统时间错误	配置 CA，校时，确认代理。
下载极慢但能连通	直连 PyPI 慢	使用可信镜像或公司内部索引。
`407 Proxy Authentication Required`	代理需要认证	在代理 URL 中提供凭据，或用公司推荐方式。

表 10-3 网络层排查。先看 `-v` 输出，不要同时改镜像、代理和证书。

10.4 一页环境快照

把下面内容保存到项目根目录的 `env-report.txt`，贴给同事或 Issue 时，对方能直接判断问题在哪一层。不要包含令牌与私有 URL。

```
# env-report.txt（示例）
OS: Windows 11 26100 / macOS 15 / Ubuntu 24.04
Python: 3.13.7
sys.executable: D:\code\demo\.venv\Scripts\python.exe
sys.prefix: D:\code\demo\.venv
sys.base_prefix: C:\Program Files\Python313
pip: 25.2 (D:\code\demo\.venv\Lib\site-packages\pip)
venv created by: py -3.13 -m venv .venv
PY_PYTHON: 3.13
PATH first entries: .venv\Scripts; C:\Windows\System32; ...
pip check: No broken requirements found.
```

为什么这四个路径最重要

`sys.executable` 说明当前解释器；`sys.prefix` 说明环境；`sys.base_prefix` 说明基础版本；pip 路径说明包装到了哪里。四者一致，环境基本就是清楚的。

TROUBLESHOOTING / 25 CASES

11

常见报错 25 例

这一章按“Windows 与 PATH → venv 与 pip → 依赖与编译 → macOS/Linux”分组。每条先给最可能的原因，再给最小修复。一次只改一个变量，避免把问题越修越乱。

按需查阅 | 25 个高频问题 | 命令可复制

11.1 Windows 与 PATH

01 “python 不是内部或外部命令”

Python 没进 PATH，或当前终端还是安装前的环境。重开终端，运行 `where.exe python`；没有输出就说明 PATH 问题。多版本场景不要急着加 PATH，先用 `py -3.13` 验证解释器是否已注册。

02 输入 python 却弹出 Microsoft Store

`WindowsApps\python.exe` 是商店别名。设置 → 应用 → 高级应用设置 → 应用执行别名，关闭 `python.exe` 与 `python3.exe`。然后重开终端。

03 “py 不是内部或外部命令”

安装时没勾选 py launcher，或只装到了当前用户但 PATH 没有 launcher 目录。重新运行安装器 → Modify → 勾选 py launcher；全用户安装时确认 `C:\Windows\py.exe` 存在。

04 装了 3.13，但 `py -3.13` 找不到

可能装的是便携版/嵌入式版，未注册；也可能安装到了另一个账户。运行 `py -0p` 看列表；用官方安装器重装并勾选 py launcher。不要只复制 python.exe 到 PATH。

05 Activate.ps1 被系统禁止运行

执行 `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`，重开 PowerShell。不要用 `Bypass` 长期放开；公司电脑按组策略处理。

06 `where python` 出现多条路径

说明多个入口共存。用 `Get-Command python -All` 看顺序；保留一个主入口，其他版本用 `py -3.x`。不要把每个版本的根目录和 Scripts 都放进 PATH。

07 安装完成，PATH 里却没有变化

安装器只修改它认为需要的 PATH；如果你取消了“Add to PATH”，它不会自动加。多版本环境这是正常的。用 `py` 调版本；只给一个主版本加入 PATH。

11.2 venv 与 pip

08 Ubuntu 上 `python3 -m venv` 报 ensurepip 不可用

缺 `python3-venv`。执行 `sudo apt install python3-venv`；如果用 3.13，则装 `python3.13-venv`。不要用 `--without-pip` 绕过。

09 创建 venv 时报权限错误

目标目录不可写，或杀毒软件锁定。换到用户目录（如 `D:\code`），不要装进 `C:\Program Files` 或 OneDrive；以普通用户运行，不要用管理员创建后再用普通用户激活。

10 CMD 里 `Activate` 不是命令

CMD 用 `.\.venv\Scripts\activate.bat`；PowerShell 用 `Activate.ps1`；Git Bash 用 `source .venv/Scripts/activate`。不要混用。

11 激活后 `pip` 还是装到全局

用 `python -m pip --version` 看路径。若不在 `.venv`，说明 `pip` 是另一个入口；改用 `python -m pip`，并检查 PATH 中是否有旧的 pip。

12 安装成功，运行仍 ModuleNotFoundError

安装与运行不是同一个解释器。分别打印 `sys.executable`；在 IDE 中确认解释器，在 Jupyter 中确认 kernel。不要重复安装同一个包。

13 把 .venv 复制到新电脑后不能用

venv 绑定绝对路径与平台二进制，不能跨机器/跨目录复制。删除后用新机器上的解释器重建：`python3.13 -m venv .venv`。

14 No module named pip

环境创建时没装 pip，或 ensurepip 被禁用。用 `python -m ensurepip --upgrade` 补装；仍失败则重建 venv，并确认 `python3-venv` 已安装。

11.3 依赖、编译与 SSL

15 Could not find a version that satisfies the requirement

包名拼错、版本不存在，或当前 Python 版本没有对应 wheel。先运行 `python -m pip index versions 包名`，再确认 `Requires-Python`；不要盲目加 `--pre`。

16 Microsoft Visual C++ 14.0 required

该包没有 Windows wheel，pip 尝试源码编译。优先升级 pip 或改用有 wheel 的版本；确实需要编译时安装 Visual Studio Build Tools 的 C++ 工作负载。

17 **SSL certificate verify failed**

公司代理/自签根证书或系统时间错误。让 IT 提供 PEM，配置 `pip config set global.cert ...`；Windows 使用系统证书库的 Python 通常更少出问题。不要关闭校验。

18 **代理配置了但 pip 仍超时**

检查 `pip config list -v` 与 `HTTPS_PROXY`；有些工具只读取环境变量，有些只读 pip 配置。用 `-v` 看实际连接地址，别同时设三套代理。

19 **两个包依赖冲突，pip 解析失败**

用 `pip check` 与报错中的版本范围定位；能拆环境就拆，不能拆就选兼容版本。不要用 `--force-reinstall` 假装解决。

20 **磁盘被 pip 缓存占满**

`python -m pip cache dir` 查看位置，`python -m pip cache purge` 清理。定期删除废弃的 `.venv`，但不要删正在使用的环境。

11.4 macOS 与 Linux

21 **macOS 上 `python3` 版本总是不对**

`which -a python3` 看顺序；Homebrew、python.org、系统 Python 可能同时存在。显式使用 `python3.13`，或把 Homebrew 的 bin 放到 PATH 前。不要改 `/usr/bin/python3`。

22 **pyenv 装了版本，但 python 还是系统的**

确认 `~/.zshrc` 有 `eval "$(pyenv init -)"`，执行 `exec zsh`；运行 `pyenv versions` 与 `pyenv which python`。VS Code 的解释器要单独重选。

23 **Linux 上 pip 提示 externally-managed-environment**

这是 PEP 668 保护。创建 venv 或用 `pipx`；不要用 `--break-system-packages` 长期破坏系统包管理。

24 **conda activate 不生效**

运行 `conda init zsh` 或 `conda init bash`，重开 shell；确认 `conda info --base`。不要同时让 pyenv 与 conda 的 shim 抢同一个 python。

25 **M1/M2 上某个包安装失败**

先看是否有 arm64 wheel；没有则用 conda-forge、Rosetta 终端或源码编译。不要用 `--no-binary :all:` 强迫所有包源码构建。

MAINTENANCE / SECURITY

12

维护、安全与迁移

Python 环境会随着系统更新、依赖升级和项目更替而变化。把“升级、备份、清理”做成固定动作，比出问题后重建更省时间。最后这一章给出最小维护框架。

预计 15 分钟 | 长期适用 | 含迁移清单

12.1 版本升级策略

Python 的升级分两类：补丁升级（3.13.7 → 3.13.8）与次版本升级（3.13 → 3.15）。前者修缺陷与安全问题，风险低；后者可能改变语法、弃用接口与 C API，需要测试。

类型	建议频率	做法
补丁升级	每季度或安全公告后	覆盖安装补丁版；重建项目环境并跑测试。
次版本升级	新项目启动时 / 年度评估	保留旧版本，新增版本并行；用 tox/CI 验证依赖。
EOL 迁移	EOL 前 6–12 个月	列出依赖的 `Requires-Python`，逐步升级；不要等到最后一刻。

表 12-1 升级策略。生产环境先测试，再切换默认版本。

- [ ] **备份锁定文件。** `requirements.txt` / `uv.lock` 先提交。
- [ ] **记录当前版本。** `python --version` 与 `pip freeze` 留档。
- [ ] **在副本上试。** 用新版本重建一个临时环境，不直接覆盖主力环境。
- [ ] **跑测试与构建。** 不要只看 `import` 成功。
- [ ] **保留回滚。** 旧解释器与旧环境至少保留一个迭代周期。
- [ ] **更新文档。** 把新版本写入 README / CI 矩阵。

12.2 供应链与镜像安全

虚拟环境隔离的是依赖，不是信任。从公共 PyPI 安装包时，仍然要面对拼写抢注、恶意包、依赖混淆与镜像篡改。

- **锁定版本：** 应用项目提交锁文件；生产部署使用 `--require-hashes` 或私有索引的校验机制。
- **核对来源：** 公司私有包使用内部索引；不要让公共镜像有机会提供同名私有包（依赖混淆）。
- **最小权限：** CI 的包发布令牌只给必要项目与短有效期；不要把令牌写进 `.pypirc` 后提交。
- **审计：** 定期运行 `pip-audit`（第三方工具）或公司安全扫描；关注 Python 与依赖的 CVE。
- **镜像：** 镜像能看到包名与下载行为；敏感项目使用公司镜像或直连，并确认 TLS。

> **不要把“内网”当成安全**
> 内网镜像如果长期不更新，可能包含已知漏洞；如果配置了 `trusted-host`，还可能被中间人替换。安全来自校验、权限与更新，而不是网络位置。

12.3 迁移、备份与清理

换电脑时，不要复制 `.venv`，也不要复制整个 Python 安装目录。正确迁移的是“声明”和“数据”，环境在新机器重建。

对象	迁移方式	说明
项目代码	Git clone / 压缩包	确保 `.venv` 未被提交。
依赖声明	`pyproject.toml` + 锁文件	新机器执行 `pip install` / `uv sync`。
解释器	重新安装	不要复制安装目录；注册表/路径会失效。
虚拟环境	**不迁移**	平台、路径、二进制都绑定本机。
密钥与配置	密码管理器 / 加密介质	`.env` 不进入 Git；生产密钥用 Secret Manager。

表 12-2 迁移清单。能在十分钟内重建环境，才算真正可复现。

清理

```
# 清理 pip 缓存
$ python -m pip cache purge

# 查看磁盘占用（macOS / Linux）
$ du -sh ~/.cache/pip ~/.pyenv/versions/* 2>/dev/null

# 删除不再使用的项目环境（确认路径后）
$ rm -rf /path/to/old-project/.venv
```

Windows 用“设置 → 应用”卸载不需要的解释器，手动删除项目下废弃的 `.venv`。不要删除 `%LOCALAPPDATA%\Programs\Python\Launcher`，除非你确定不再使用 `py`。

END OF MAIN TEXT

到这里，解释器安装、多版本入口、venv、pip、IDE、验收与维护已经闭环。附录 A 是命令速查，附录 B 是两个可直接使用的初始化脚本，附录 C 是版本与资源表。

APPENDIX A / CHEAT SHEET

A

命令速查表

按“解释器 → venv → 依赖 → 诊断”分组。尖括号表示必须替换的占位符；命令默认在项目根目录执行。

可打印 30 条命令 跨平台

A.1 解释器与入口

命令	作用
`py -0p`	Windows 列出已注册解释器与路径。
`py -3.13 --version`	调用指定版本。
`py -3.13 -m venv .venv`	用指定版本创建环境。
`which -a python3`	macOS/Linux 查看所有候选入口。
`python3.13 -m venv .venv`	用版本化命令创建环境。
`python -c "import sys; print(sys.executable)"`	查看当前解释器绝对路径。
`python -m site`	查看 site-packages 与用户目录。

A.2 venv 激活

命令	Shell / 作用
`.\.venv\Scripts\Activate.ps1`	PowerShell 激活。
`.\.venv\Scripts\activate.bat`	CMD 激活。
`source .venv/Scripts/activate`	Git Bash 激活。
`source .venv/bin/activate`	macOS/Linux 激活。
`deactivate`	退出当前环境。
`.venv/bin/python app.py`	不激活直接运行。
`.\.venv\Scripts\python.exe app.py`	Windows 不激活运行。

A.3 pip 与依赖

命令	作用
`python -m pip install -U pip`	升级 pip。
`python -m pip install -r requirements.txt`	安装依赖。
`python -m pip freeze > requirements.lock.txt`	冻结精确版本。
`python -m pip install -e ".[dev]"`	可编辑安装并带开发依赖。
`python -m pip check`	检查依赖冲突。
`python -m pip list --outdated`	查看可升级包。
`python -m pip cache purge`	清理下载缓存。
`python -m pip config list -v`	查看索引/代理/证书配置。

A.4 诊断与多版本

命令	作用
`python -m pip --version`	确认 pip 属于哪个环境。
`python -m ipykernel install --user --name ...`	注册 Jupyter kernel。
`jupyter kernelspec list`	查看 kernel。
`tox` / `nox`	多版本测试矩阵。
`uv python install 3.13`	uv 安装解释器。
`uv venv --python 3.13`	uv 创建环境。
`conda create -n py313 python=3.13`	conda 创建环境。
`conda env export --from-history`	导出 conda 环境。

表 A-1 常用命令速查。所有安装命令都应指向项目环境，而不是全局 Python。

APPENDIX B / BOOTSTRAP SCRIPTS

B

初始化脚本模板

把“建环境、升 pip、装依赖、打印解释器”写成脚本，能让新电脑初始化从十几分钟降到一分钟。下面两个模板只使用标准工具，复制后按项目修改。

PowerShell Bash 可审计

B.1 Windows PowerShell

```powershell
# init-python.ps1
param(
    [string]$Version = "3.13",
    [string]$Venv = ".venv"
)

$ErrorActionPreference = "Stop"
Set-Location -LiteralPath $PSScriptRoot

if (Test-Path -LiteralPath $Venv) {
    throw "$Venv already exists. Remove it explicitly before recreating."
}

Write-Host "Creating $Venv with Python $Version ..."
py "-$Version" -m venv --upgrade-deps $Venv

$python = Join-Path $Venv "Scripts\python.exe"
& $python -m pip install --upgrade pip

if (Test-Path "requirements.txt") {
    & $python -m pip install -r requirements.txt
}

& $python -c "import sys; print('python:', sys.version); print('exe:', sys.executable)"
Write-Host "Done. Activate with: .\$Venv\Scripts\Activate.ps1"
```

使用方式：把脚本放在项目根目录，打开 PowerShell，执行 `.\init-python.ps1`；需要 3.15 时执行 `.\init-python.ps1 -Version 3.15`。脚本不会自动删除已有环境，避免误删依赖。

执行前先读脚本

如果 PowerShell 阻止运行本地脚本，使用 `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`，不要直接 `Bypass`。脚本应纳入版本控制，任何改动都走评审。

B.2 macOS / Linux Bash

```
#!/usr/bin/env bash
set -euo pipefail

VERSION="${1:-3.13}"
VENV=".venv"
cd "$(dirname "$0")"

if [ -e "$VENV" ]; then
  echo "$VENV already exists; remove it explicitly first." >&2
  exit 1
fi

echo "Creating $VENV with Python $VERSION ..."
"python$VERSION" -m venv --upgrade-deps "$VENV"

"$VENV/bin/python" -m pip install --upgrade pip
if [ -f requirements.txt ]; then
  "$VENV/bin/python" -m pip install -r requirements.txt
fi

"$VENV/bin/python" -c 'import sys; print("python:", sys.version); print("exe:", sys.executable)'
echo "Done. Activate with: source $VENV/bin/activate"
```

保存为 `init-python.sh`，执行 `chmod +x init-python.sh`，再运行 `./init-python.sh 3.13`。脚本使用 `python$VERSION`，所以解释器必须已经安装并能以版本化命令找到。

B.3 requirements 模板

```
# requirements.in（直接依赖）
requests>=2.32,<3
python-dotenv>=1.0,<2

# requirements-dev.in（开发依赖）
pytest>=8,<9
ruff>=0.9,<1
```

```
# 生成锁定文件
$ python -m pip install pip-tools
$ pip-compile requirements.in -o requirements.txt
$ pip-compile requirements-dev.in -o requirements-dev.txt
$ python -m pip install -r requirements.txt -r requirements-dev.txt
```

应用部署用锁定文件；库项目在 `pyproject.toml` 中写范围，不要把开发机的 `pip freeze` 直接发布。

APPENDIX C / REFERENCE

C

版本、资源与术语

最后放一张版本支持表、一份官方链接和一份术语表。版本会变化，遇到不确定的地方，以 python.org 与工具官方文档为准。

2026-10 支持周期 术语 16 条

C.1 版本支持表

版本	首次发布	状态	安全支持至
3.15	2026-10-09	bugfix	2031-10
3.14	2025-10-07	bugfix	2030-10
3.13	2024-10-07	security	2029-10
3.12	2023-10-02	security	2028-10
3.11	2022-10-24	security	2027-10
3.10	2021-10-04	**EOL**	2026-10-01 已结束

表 C-1 支持周期（截至 2026-10）。补丁版本号会继续更新，EOL 日期以 PEP 为准。

C.2 官方资源

Python 下载
https://www.python.org/downloads/
Windows/macOS 安装器、源码与所有版本。

Python 文档
https://docs.python.org/3/
标准库、venv、pip 引导与教程。

venv 文档
https://docs.python.org/3/library/venv.html
创建参数、pyvenv.cfg 与行为说明。

pip 文档
https://pip.pypa.io/en/stable/
安装、配置、缓存、依赖解析。

Python 版本状态
https://devguide.python.org/versions/
官方支持周期与 EOL 表。

uv 文档
https://docs.astral.sh/uv/
解释器、项目、锁文件与 CI。

pyenv
https://github.com/pyenv/pyenv
安装、shell 初始化与编译依赖。

conda 文档
https://docs.conda.io/
环境、包管理与 environment.yml。

PyPI
https://pypi.org/
包索引；安装前确认项目主页与版本。

C.3 路径速查

对象	Windows	macOS / Linux
解释器	`C:\Program Files\Python313\python.exe`	`/opt/homebrew/bin/python3.13`
py 启动器	`C:\Windows\py.exe`	不使用（用 pyenv/版本化命令）
项目环境	`.venv\Scripts\python.exe`	`.venv/bin/python`
用户配置	`%APPDATA%\pip\pip.ini`	`~/.config/pip/pip.conf`
pip 缓存	`%LOCALAPPDATA%\pip\Cache`	`~/.cache/pip`

表 C-2 常见路径。路径会随安装方式变化，用 `python -m pip cache dir` 等命令查实际值。

C.4 术语表

术语	解释
解释器	实际执行 Python 代码的程序，例如 `python3.13.exe`。
venv	标准库虚拟环境；隔离包与入口，不复制标准库。
pyvenv.cfg	虚拟环境的配置文件，记录基础解释器与版本。
site-packages	第三方包安装目录；venv 内有独立副本。
PATH	操作系统查找命令的目录列表，顺序决定入口。
py 启动器	Windows 按版本号选择解释器的工具。
pyenv shim	pyenv 放在 PATH 前的转发脚本，按目录选择版本。
wheel	预编译的二进制分发包；没有 wheel 时 pip 可能源码编译。
sdist	源码分发包，安装时需要构建步骤。
PEP 668	标记系统 Python 为 externally managed，阻止直接 pip 安装。
lock file	记录完整依赖解析结果，保证可复现。
editable install	`pip install -e`，源码改动立即生效。
kernel	Jupyter 执行代码的解释器会话。
conda env	conda 管理的独立环境，可包含非 Python 依赖。
EOL	End of Life，停止安全支持。
reproducible	同一份声明能在另一台机器重建相同环境。

表 C-3 阅读文档与排查时的高频术语。

COLOPHON

版本：1.0 · 更新：2026 年 10 月 · 基线：Python 3.15.2 · 适用：Python 3.10+
排版：A4 / 微软雅黑 / Consolas；生成方式：HTML + Paged.js + Chromium PDF。

AFTER SETUP

版本可以很多，入口只能有一个。

知道每条命令实际调用谁，每个项目有自己的环境，换一台机器能在十分钟内重建。做到这三点，Python 环境就不再是玄学。

01 **解释器：看得见、说得清**

02 **项目：一个目录一个 .venv**

03 **依赖：有声明、有锁定**

Python 多版本与虚拟环境

v1.0 · 2026 年 10 月 · 基于 Python 3.15.2

py.